Transformation Techniques in the Model-Driven Development Process of UWE

Nora Koch
Ludwig-Maximilians-Universität
Oettingenstr. 67, 80538 München
and FAST GmbH
Arabellastr. 17, 81925 München,
Germany
kochn@pst.ifi.lmu.de

ABSTRACT
Development of Web software is still an inefficient and error-prone process. We need integrated techniques and tool support for automated generation of Web systems. The goal of model-driven development (MDD) is to tackle these problems introducing a higher level of abstraction by defining metamodels and model transformations rules. We present the development process of the UML-based Web Engineering (UWE) approach defined as an MDD approach and focus on the model transformation aspects of the process.

Categories and Subject Descriptors
I [**Computing Methodologies**]: I6 Simulation and Modeling, I6.5 Model Development – *modeling methodologies.*

General Terms
Design, Languages, Standardization.

Keywords
Model-Driven Development, Model, Modeling Language, MDA, Transformation, Transformation Language, UML, UWE, Web Engineering.

1. INTRODUCTION
Software development techniques are continuously evolving with the goal of solving the main problems that still affect the building and maintenance of software systems: time, costs and error-proneness. Regarding the development process two important tendencies can be observed: agile and model-driven development.

Model-driven development (MDD) [4] focuses on the construction of models, specification of transformation rules, tool support and automatic generation of code and documentation. Agile development [1] instead focuses on the stakeholder's activities. The central idea of MDD is to separate the platform independent design from the platform specific implementation of applications delaying as much as possible the dependence on specific technologies. Therefore, MDD advocates the construction of platform independent models and the support of model transformation. Consequently, the software development process can be viewed as a chain of model transformations.

Web Engineering is a concrete domain where MDD can be helpful, particularly in addressing the problems of evolution and adaptation of Web software to continuously emerging new platforms and changes in technologies. During the last years the Web engineering community has proposed several languages, architectures, methods and processes for the development of Web applications. In particular, methods for modelling such systems were developed, such as Hera [9], OOHDM [29], OO-H [10], UWE [18], WebML [5], and W2000 [2]. They specify how to build navigation or adaptation model. However, the model transformation aspects were not addressed explicitly.

We present the MDD process of the UML-based Web Engineering (UWE) approach and focus on the model transformation aspects of this process. The UWE process covers the whole development life cycle of Web systems from the requirements specification to code generation. The difference to other approaches in the Web domain, as far as we know, is the specification of models and transformation rules mainly using forthcoming transformation languages for each step of the development chain.

The most well known MDD realization is the Model-Driven Architecture (MDA) of the OMG [23]. We will use MDA to illustrate our approach, as UWE already uses other OMG standards (UML [27], XMI [28], MOF [24], OCL [25], QVT [26]), and which are the pillars on which the MDA process is based on.

The remainder of this paper is structured as follows: Sect. 2 gives a brief overview of the relevant MDD concepts. Sect. 3 presents the UWE process and the description of the UWE models. An analysis of the transformations that are applied to the source models in each step of the UWE process is presented in Sect.4. Sect. 5 provides an overview of related work. Finally, in Sect. 6 we present some conclusions and outline our work in progress and future plans on the implementation of the model transformations.

2. BASIC MDD CONCEPTS
The idea behind MDD is that modeling and transforming is a better foundation for developing and maintaining systems than programming [22]. A model-driven approach requires languages for the specification of models, the definition of transformations, and the description of metamodels. The concrete techniques developed so far supporting the MDA approach of the OMG [23] include the UML, QVT and MOF.

There are still problems with appropriate tool support and exchange formats, which are needed for a seamless implementation of the process, but we are observing how

research, industry interest and standardization efforts are moving to support the complete MDD process.

2.1 Types of Models

A *model* of a system is a specification of that system and its environment for some purpose. Models consist of a set of elements with a graphical and/or textual description. The idea of MDD is creating different models of a system at different levels of abstraction and linking them together to produce the implementation of the system. MDA suggests building computational independent models (CIM), platform independent models (PIM) and platform specific models (PSM) corresponding to different levels of abstraction or viewpoints [23]. We will use these types to classify the UWE models in Sect. 3.

The *computational independent viewpoint* focuses on the environment of the system, and the requirements the user has on the system; the description provides what the system is expected to do. The details of the structure and processing are hidden or yet undetermined. A computational independent model is sometimes called a domain model or a business model. It should be traceable from the PIM and PSM models that implement the CIM.

The *platform independent viewpoint* focuses on the operation of a system while hiding the details for a particular platform. It shows the part of the complete specification that does not change from one platform to another.

The *platform specific viewpoint* combines the platform independent viewpoint with additional features on the specific platform. .

The objective is to postpone in the development process the creation of models that take into account technological aspects of a platform as much as possible. The main advantage is to be able to react efficiently and with low costs to technology changes.

2.2 Types of Transformations

Model transformation is the process of converting one or more models – called source models – to one output model – the target model – of the same system [23]. Note that the model transformation result is exactly one model. We distinguish the following aspects of transformations: Viewpoint of source and target model, level of abstraction, use of marks, execution and implementation types. We use these aspects as classification criteria for the transformations of the UWE process presented in Sect. 4.

2.2.1 Viewpoint of Source and Target Models

In a model-driven development process model transformations can be of type CIM to PIM, PIM to PIM, PIM to PSM and PSM to code. A computational independent model can be refined, i.e. a CIM can be mapped to another CIM, in the same way PIMs can be refined. Note that transformations from PIM to CIM, PSM to CIM, and PSM to PIM are not possible.

2.2.2 Transformation Levels of Abstraction

Transformations can be applied at different levels: metamodel, model or instance. Transformations, however, can consist of some combination of the above approaches.

A *transformation at metamodel level* is the specification of certain types of source models that are converted to another type of target models. The transformation rules that result from this metamodeling mapping can be applied then to all source model instances as represented in the pattern of Bezivin [4] in Figure 1.

A *transformation at model level* consists on the identification of model elements that will be transformed in a certain way and a *transformation at instance level* consists on the identification of specific objects that will be transformed in a certain way.

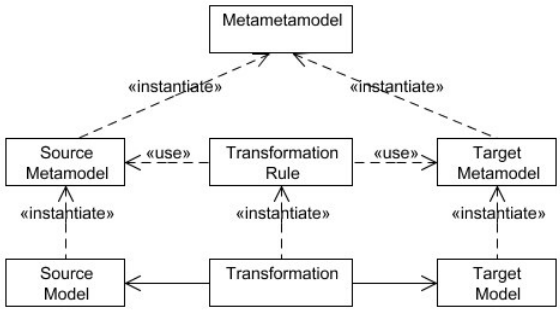

Figure 1: Model transformation pattern [4]

2.2.3 Use of Marks and Additional Information

Transformation rules rely on certain *marks* (types, patterns, templates or UML profile elements) in order to select the elements to which rule applies [23]. These marks can be part of the elements or take the form of additional input that does not pollute the source models, i.e. non-intrusive or lightweight extension to models. Examples of marks provided by the model itself are *types* (class or association) and *stereotypes of UML profiles*. In addition, *patterns* identifiable in the source model can also be used as a mark in a transformation rule, as a certain combination of modeling elements.

Other marks instead are only required for the mappings. They do not need to be integrated in the source model [22], such as selection of certain classes or states. This kind of marks are kept in separate *marking models* and combined with the source models during the mapping process. *Templates* are other external providers of input for transformations. They are like patterns but may include more detailed information to guide the transformation.

Additional information can be used to guide the transformation. Often it is drawn from the knowledge on the application domain or knowledge on the platform of the designer. For example a particular architecture style or pattern may be specified.

2.2.4 Merge Transformations

According to the number of source models involved in the mapping process a transformation is named simple or a merge.

2.2.5 Execution of Transformations

Transformations are classified in manual, semi-automatic and automatic based on the involvement of a person and/or a system in the execution of the mapping rules.

A transformation is automatic if it does not require any decision from the user of the system. On the contrary, the transformation is manual if the user takes the decision of which elements will be transformed. A model-driven process aims to define

transformations rules that allow for automatic model transformations. If not possible in some cases, semi-automatic transformation rules will be introduced.

2.2.6 Implementation of Transformations

Transformation rules can be implemented in (1) general programming language as Java, i.e. hard coded in specific tools, such as the case of ArgoUWE [14], or (2) languages for transformations as ATL [13] or QVT [26]. Transformations are often based on invariants and pre and post conditions specified in languages such as OCL [25].

3. UWE PROCESS AND MODELS

The UWE approach comprises a UML Profile for modeling Web systems, a process and tool support for the development of Web systems. For modeling aspects of UWE and UWE CASE tool we refer the reader to [18], [16], [1], [6] and [14].

The UWE process is a model driven development process following the MDA principles and using the OMG standards ([23], [24], [25], [26], [27], [28]). The process consists of models and model transformations. The specification of these models and model transformations is supported by metamodels and transformations rules.

3.1 Process Overview

The main characteristic of the UWE process is the systematic, semi-automatic, model-driven and transformation-based (Sect. 4) support of the development of Web systems.

The UWE process is depicted in Figure 2 as a stereotyped UML activity diagram ([21]). Models are depicted as objects part of the object flow and transformations as stereotyped activities (special circular icon). A chain of transformations then defines the control flow.

The process starts with the business model (CIM) level defining a requirements model. Platform independent design models (PIMs) are derived from these requirements. The set of design models represents the different concerns of the Web applications. It comprises the content, the navigation, the business processes, the presentation, and the adaptation of the Web system. The different design models are not depicted in the overview.

Design models are afterwards integrated mainly for the purpose of e.g. verification into a big picture model [16]. A merge with architectural modelling features results in an integrated PIM model covering functional and architectural aspects. Finally, the platform specific models (PSMs) are derived from the integration model from which programming code can be generated. The aim of such an MDD process is automatic model transformation in each step based on rule transformations.

3.2 Models in UWE

A set of models of a Web system is built during the UWE development process. Each model belongs to one of the three viewpoints described in Sect. 2.1, i.e. CIM, PIM or PSM (see Figure 2). We illustrate models and model transformations by means of a music Web portal example, inspired by `www.mp3.com`, which provides albums for download. Information about singer, composer, and publisher are available for free, instead only registered users can search albums and download them if they have enough credit on their prepaid account.

An outline of the models used in UWE is given in the following. Examples of a simple music portal are also shown. For further details on UWE models refer to [1], [6], [14], [16], [18] and [19].

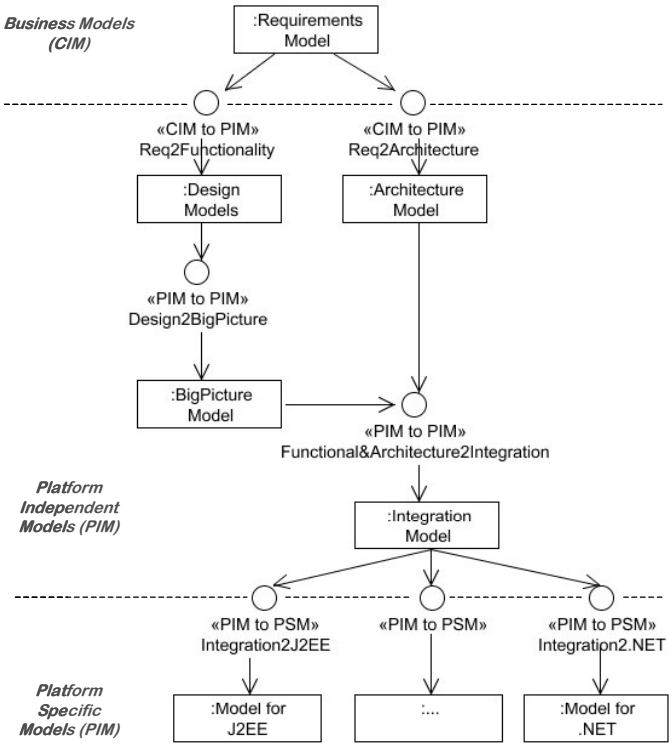

Figure 2: UWE Process Overview

3.2.1 Requirements Model

The overall objective of modeling the requirements is the specification of the functionality of the system as a computational independent model (CIM). The specific objectives for Web systems become: (1) the specification of content requirements, (2) the specification of the functional requirements in terms of navigation needs and business processes, and (3) the definition of interaction scenarios for different groups of Web users.

UWE models requirements with UML use case diagrams and UML activity diagrams. UWE distinguishes two types of use cases: navigation use cases and use cases describing Web business processes. The latter – following UWE's recommendation – should be further detailed with activity diagrams. UWE uses the UML profile for Web requirements (WebRE) defined by Escalona & Koch [6], which comprises stereotyped use cases, activities and objects providing modeling elements with Web domain specific semantics. Figure 3 depicts the use case diagram for the music portal and Figure 4 shows the activity diagram for the *Download album use* case.

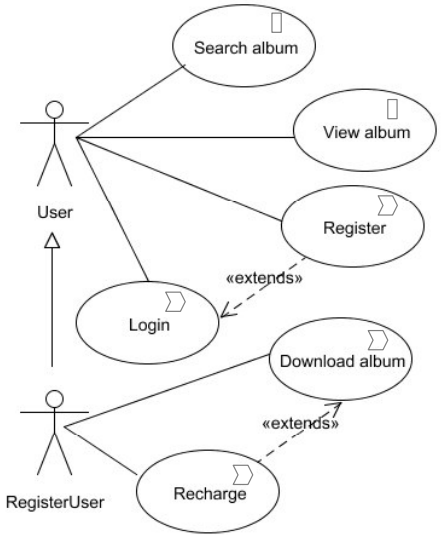

Figure 3: Use case diagram (CIM)

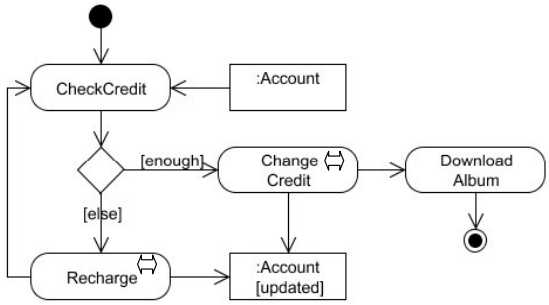

Figure 4: Activity diagram for a (simplified) download album use case (CIM)

3.2.2 Design Models

At design level UWE follows the separation of concerns widely applied in Web engineering. We build separate models for content, navigation and presentation aspects of Web systems using UML class diagrams for the visual representation [18]. We supplement them with an additional process model for transactional Web applications, and an adaptation model for personalized and context-dependent systems. UWE defines Web domain specific modeling elements e.g. navigation class and menu for the navigation model, and presentation class and anchor for the presentation model.

The UWE profile provides the corresponding stereotypes. Figure 5 and Figure 6 depict the content and navigation model of the music portal. Navigable nodes are represented by instances of the metaclass *NavigationNode* such as *NavigationClass*, *Menu* and *ProcessClass* (stereotypes that extend UML class). Links between navigation nodes are represented by instances of *NavigationLink* and *ProcessLink*. In addition, navigation paths are structured by instances of special types of access primitives such as *Index, Query* and *GuidedTour*. Indexes represent choices among instances of a navigation class, menus (like *MainMenu*) in contrast represent choices among instances of navigation nodes of different types. A Query (like *SearchAlbum*) models a search action in the Web application, where a user can enter a term and select from the matching results.

Process models are visualized as UML 2.0 activity diagrams (Figure 7). Actions (like *FindUser*) model the actions the user and the system must carry out to complete the business process.

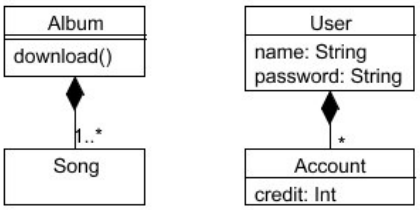

Figure 5: Content and user model (PIM)

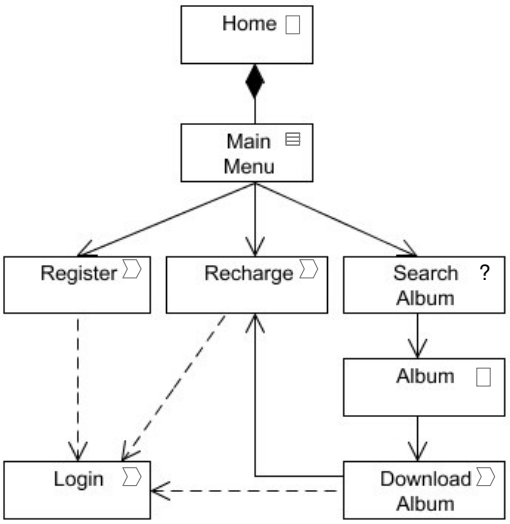

Figure 6: Navigation model (PIM)

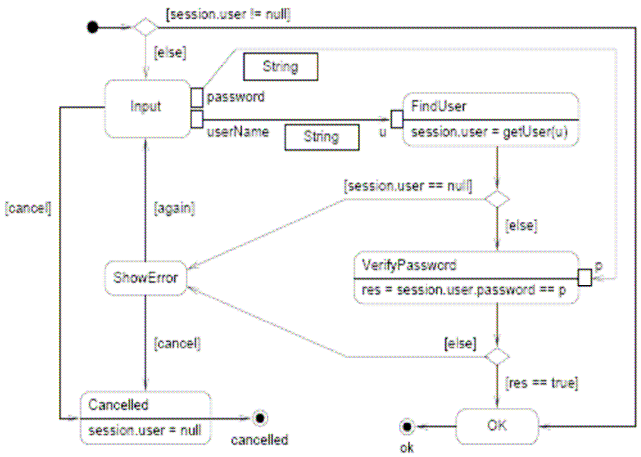

Figure 7: Business process Login (PIM)

UWE proposes to build a presentation model to sketch the layout of the Web application. It uses the UML composition notation for classes, i.e. containment represented by graphical nesting of the symbols. This kind of representation is appropriate for modeling user interfaces as it allows for spatial ordering, but has the problem that standard case tools cannot manage it. For adaptation

models the UWE profile includes stereotypes for different node and link adaptation. The diagrammatic technique used by UWE is aspect oriented modeling (AOM), extending the UML with concepts such as aspect, pointcut and advice to support AOM [1]. Due to space restrictions we do not give further details to presentation and adaptation models in this paper; the reader is referred to [17] and [18].

UWE proposes the generation of an integrated model that merges the separate concerns of a Web system into a big picture. Using UML state machines as the results of the integration process offers the possibility of applying formal techniques for validation, like model checking [15]. Currently the big picture is the result of the integration of the UWE content, navigation and business logic models, but it can be easily be extended to include features like access control [33] and adaptation. Figure 8 shows the states *Home* and *Song*, both states are part of the "Big Picture" model.

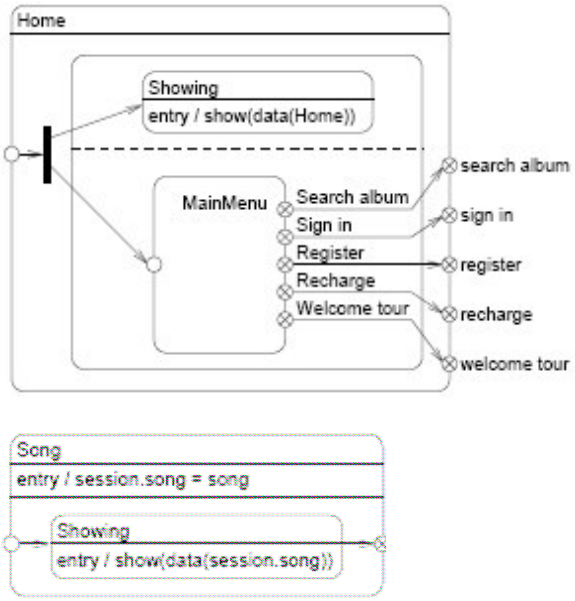

Figure 8: States Home and Song of the "big picture" (PIM)

3.2.3 Architecture and Implementation Models

Additional information on architectural styles can be merged at different steps in he UWE MDD process. Following the WebSA approach we propose in [21] to integrate functional design models and architecture models in a very early development phase. Such an approach is shown in the UWE process overview depicted in Figure 2. Architecture models in the WebSA approach are specified as platform independent models (PIMs). Knapp and Zhang suggest in [16] to merge architecture models with the big picture model, i.e. with the result of the already integrated model of the different concerns (content, navigation and business logic). A third alternative is to introduce the architectural information in the generation of platform specific models.

4. MODEL TRANSFORMATIONS IN UWE

Model transformations are based on the definition of transformation rules, which are defined whenever possible at metamodel level and written as expressions of transformation languages. Hence, we need the specification of the metamodels of both the source and the target of the transformation. In addition, to the UWE metamodel we use the metamodel for Web requirements engineering (WebRE) [6] and the Web Software Architecture metamodel (WebSA) [21] that are MOF-compliant metamodels.

Transformations are grouped in those used to build the design models, those needed to generate the big picture and the integration model, and finally transformations for the generation of implementation models and code. We summarize the characteristics of each transformation in Table 1 based on the criteria defined in Sect.2.2.

4.1 Building Design Models

The first model transformation step of the UWE process consists of the mapping of the Web requirements models to the UWE design models [19]. The design models are the content, navigation, process, presentation, and adaptation model. There exists a set of dependencies among these design models themselves that allow for creation of other models or refinement of models. For example a first approach of the navigation model is obtained from the content model, which can then be further refined.

Transformations rules are defined as mappings from metamodel WebRE to the UWE metamodel and among UWE metamodels. Figure 9 shows for example, how the model transformation pattern of Figure 1 is applied to the UWE process using the standard Query View Transformation Language (QVT, [26]).

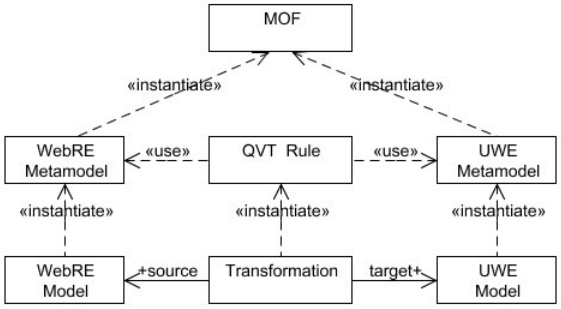

Figure 9: Model transformation pattern for metamodels WebRE and UWE

In the UWE process the transformation *requirements to content* allows for the construction of the content model, the transformations *content to navigation, requirements to navigation* and *navigation refinement* are used to build the navigation model. The presentation model is built in at least two iterations: it is created with the former *navigation to presentation* and refined by *style adjustments*. Last but not least the adaptation model can also be extracted from the functional requirements models, and the architecture models from the non-functional requirements.

Transfor-mation	Req2 Content	Req2 Architec-ture	Content2 Navigation	Navigation Refinement	Req2 Navigation	Navigation2 Presentation	Style Adjust-ment	Design2 BigPicture	Integrating Architecture	Integration2 J2EE
Type	CIM to PIM	CIM to PIM	PIM to PIM	PIM to PIM	CIM to PIM	PIM to PIM	PIM to PIM	PIM to PIM	PIM to PIM	PIM to PSM
Level of abstraction	metamodel	-	model	metamodel	metamodel	metamodel	model	metamodel	metamodel	metamodel
Source/ Target	simple	simple	simple	simple	Merge	simple	merge	merge	merge	merge
Marks	WebRE profile	-	UWE profile & navigation relevance	UWE profile & patterns	WebRE profile	UWE profile	style guide	patterns and marks	UWE & WebSA profile	patterns
Techniques	QVT	-	Java (OCL) ATL	Java (OCL)	QVT	Java (OCL) ATL	Java	graph transfor-mations	QVT	QVT, ATL
Execution	automatic	manual	semi-automatic	automatic	Automatic	automatic	automatic	automatic	automatic	automatic

Table 1: Characteristics of transformations in the UWE model-driven development process

4.1.1 Transforming Requirements to Content

Activities, such as browse, search or transactions in the Web are related to objects that are either required as input or produced as results. These objects can be included in activity diagrams by means of object flows. In the particular case of modeling Web systems requirements, objects are used to indicate the need to include certain content information in the Web application (Figure 10).

Figure 10: From requirements to content model

We use the QVT language to specify the transformation from elements of the requirements model to elements of the content model (*Req2Content*). The transformation is specified at metamodel level, i.e. the transformation rule defines the mapping of the stereotyped classes «content» of the WebRE metamodel to classes of the UWE content model. For example the QVT specification of the transformation is shown in Figure 11. For further details of the specification the reader is referred to [19]. The characteristics of the *Req2Content* transformation are summarized in Table 1.

```
transformation ReqContent2ContentClass (webre:WebRE, uwe:UWE) {
    top relation R1 {
          checkonly domain webre  c:Content { name = n };
          enforce domain uwe cc: Class { name = n };  }
    top relation R2 {
           cn: String;
    checkonly domain webre p: Property { namespace=c:
                        Content {}, name =  cn};
           enforce domain uwe p1:Property { namespace = cc: Class{};
                        name = cn}
           when {R1 (c,cc); }
    }
}
                        name = cn}
           when {R1 (c,cc); }
    }
}
```

Figure 11: Transformation requirements elements to content model (QVT textual notation)

4.1.2 Transforming Requirements to Architecture

A mapping of non-functional requirements to architectural model elements is subject of future work. Currently, the designer performs the transformation manually. A metamodel of the non-functional requirements for Web applications is still missing.

4.1.3 Transforming Content to Navigation

In UWE a first approach for the navigation model (see Figure 12) is generated based on classes of the content model marked as navigation relevant. Hence, the transformation *Content2Navigation* is defined at model level. From one content model different navigation views can be obtained, e.g. for different stakeholders of the Web system like anonymous user, registered user and administrator [18].

The generation of each navigation view requires a set of marks on elements of the content model, which comprise a so-called *marking model*, which is kept separately from the content model. The development process cannot be completed in an automatic way, as the designer has to take the decision about the "navigation relevance" marks. Once the marks have been set, the transformation defined as an OCL constraint and implemented in Java in the CASE tool ArgoUWE [14] is applied.

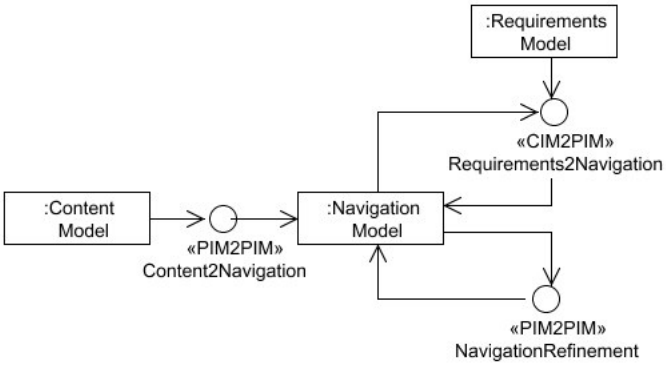

Figure 12: Transformations to build navigation model

4.1.4 Adding Requirements to Navigation

The requirements model contains information that is useful for the enrichment of the navigation model. For example, UWE distinguishes in the requirements model among different types of navigation functionality: browse, search and transactional activities. On the one side, *Browse* actions can be used to verify the existence of a navigation path between source and target nodes. On the other side, an action of type *Search* for example, indicates the need of a *Query* in the navigation model in order to allow for user input of a term and the system responding with a resulting set matching this term. Figure 13 shows the Search2Query transformation specified in the QVT graphical notation [19].

The transformation *Req2Navigation* is of type merge and is based on the WebRE profile (see Table 1). Figure 12 shows that the transformation rule *Req2Navigatio*n can be applied after the transformation rule *Content2Navigatio*n, but there is no restriction related to the order in which the *Req2Navigatio*n rule and the *NavigationRefinement* has to be applied.

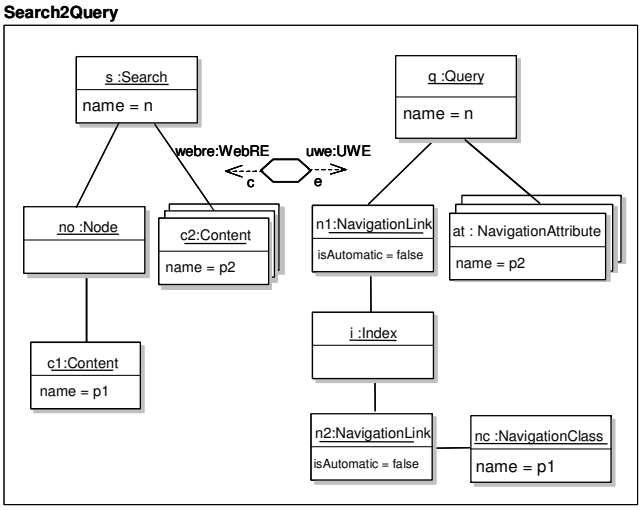

Figure 13: Search2Query transformation (QVT graphical notation)

4.1.5 Refining the Navigation Model

The navigation model generated on the content model contains itself valuable information that allows for reasoning and improving the navigation model [12]. The following constrains (informally described) define e.g. such transformation rules:

1. An index is added for all associations of the navigation model that have multiplicity greater than one at the directed association end.
2. All associations that have at least one outgoing association require a menu class with menu items defined on basis of the association ends of the associations.

These transformations are defined as OCL constraints in UWE and implemented in Java in the CASE tool ArgoUWE [14]. See Table 1 for the characteristics of these transformation rules.

4.1.6 Transforming Navigation to Presentation and Adjusting to the Presentation Style.

Presentation elements are generated based on navigation elements of the navigation model and merged then with style guide information (Figure 14). For example for each link in the navigation model an adequate anchor is required in the presentation model The main difficult is the introduction of the look and feel aspects.

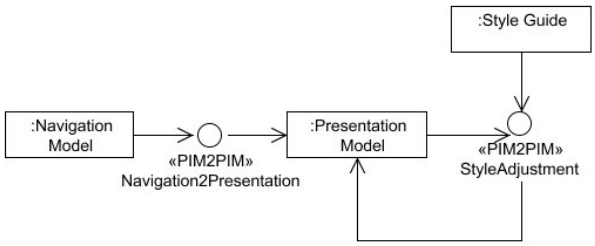

Figure 14: Transformations to build presentation model

ArgoUWE implements the *Navigation2Presentation* rule in Java. *Style2Adjustment* rules are not implement by the time this work is written. Table 1 characterizes both, the *Navigation2 Presentation* and the *Style2Adjustment.*

4.2 Creation of an Integrated Model

The aim of this phase in the UWE MDD process is the creation of one model that allows both seamless creation of platform specific models (PSMs) and validation of correctness of the models by model checking. The UWE process comprises two main integration steps: the integration of all functional models and the integration of functional and non-functional aspects; the latter related to architectural design decisions.

4.2.1 Building the "Big Picture"

Though from different viewpoints, the different design models represent the Web application as a whole. They are integrated into another platform independent model that we call the big picture (Figure 15). The big picture is used to validate the interaction of the separated models using model checking and to generate the Web application automatically. The target model is a UML state machine, representing the navigation structure and the business processes of the Web application. The big picture model can be checked by the tool Hugo/RT – a UML model translator for model checking, theorem proving, and code generation [16].

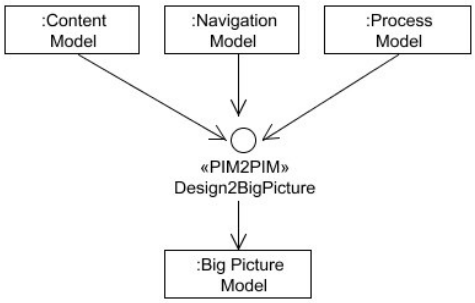

Figure 15: Transformations to build "big picture" model

The transformation rules *Design2BigPicture* form a metamodel-based graph transformation system. An example of the graph transformation of a navigation node to a navigation state in the big picture is depicted in Figure 16.

Source models are the content, business process and navigation models of UWE. Big Picture transformation rules are defined within the scope of UWE as graph transformation rules and work in progress is the implementation of these transformation rules in AGG [30] an appropriated tool for graph transformations (non-Web specific). Other characteristics of the model transformation *Design2BigPicture* are outlined in Table 1.

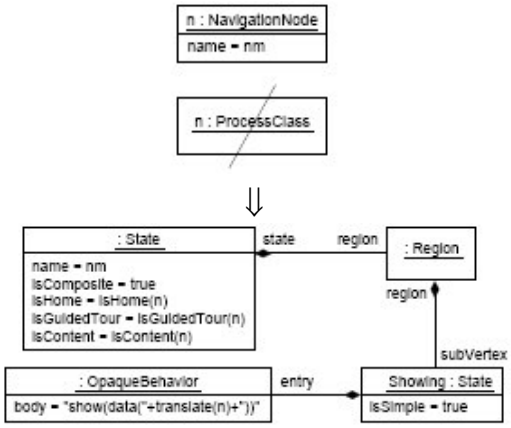

Figure 16: Mapping navigation node to state in "big picture" (graph transformation)

4.2.2 Integrating Architectural Features

Functional models defined so far (e.g. navigation, presentation, process) can be merged with architecture models (defined as PIMs) as shown in Figure 17. For example, with the elements of the WebSA models providing a layer-view and a component-view of the architecture, which are also defined as PIMs. Transformation rules are defined based on the UWE and WebSA metamodels (for further details see [21]). The characteristics of the rules of type *IntegratingArchitecturalFeatures* are outlined in Table 1.

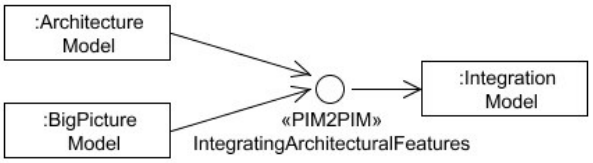

Figure 17: Integration of architectural features

4.3 Generation of Platform Specific Models and Code

To transform technology independent models into platform specific models additional information of the platform is required. It can be provided as an additional model or it is implicitly contained in the transformations. For mappings from UWE design models (PIMs) to PSMs for Web applications (Figure 18) we performed a set of experiments with the recently developed model transformation languages. The Query-View-Transformations languages used are the Atlas Transformation Language (ATL) [13], QVT-P and QVT [26]. For example, the transformation depicted in Figure 19 tackles the generation of J2EE elements from Server Pages of the Integration Model. The rule is written in QVT-P language.

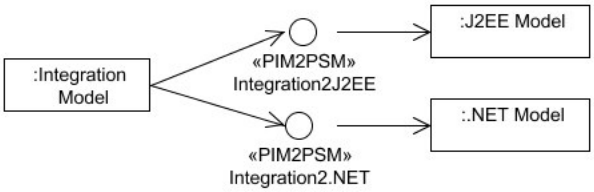

Figure 18: Generation of platform specific models

```
relation ServerPage2J2EE {
   domain { (IM.IntegrationModel) [ (ServerPage) [name=nc,
      services = {(WebService) [name=on, type=ot]}, views = {(View)
[name = vn]}]] }
domain { (JM.J2EEModel) [ (JavaServerPage) [name=nc,
forms = {(Form) [name=on, type=ot]}, beans = {(JavaClass) [name =
vn]}]] }
when { services -> forAll (s | WebService2Form (s, F1set.toChoice()) )
views-> forAll (v | View2Bean (v, J1set.toChoice()) )) }}
}
```

Figure 19: Generation of J2EE model elements based on the integration model (QVT-P language)

Another example is shown in Figure 20. The ATL code exemplifies a transformation rule that maps the element *Anchor* of the UWE presentation model to a JSP element. Note that the transformation rule also involves elements of the navigation model (*NavigationLink*) and content model (*ContentNode*).

```
rule Anchor2JSP {
from uie : UWE!Anchor (
  to jsp : JSP!Element (
    name <- 'a',
    children <- Sequence { hrefAttribute, contentNode } ),
  hrefAttribute : JSP!Attribute (
    name <- 'href',
    value <- thisModule.createJSTLURLExpr(
                    uie.navigationLink.target.name, 'objID' ) ),
  contentNode : JSP!TextNode (
                    value <- uie.name )
}
```

Figure 20: Generation of JSP elements based on the presentation model (ATL language)

5. RELATED WORK

The MDD approach of UWE focuses on model transformations defined at metamodel level and specified in general purpose transformation languages, such as QVT and graph transformations. The aim is to cover the whole development

lifecycle. Some of UWE´s transformation rules still are hard coded but we are redefining them based on the MDA paradigm.

Engels et al [8] and Varró and Pataricza [31] use graph transformations for model transformations. Their approaches do not consider the Web domain specific characteristics as they define a generic approach with focus on formal definition of transformation semantics. Baresi and Mainetti [2] propose to use rule transformation techniques for the verification of correctness and adaptability of models designed by W2000. The approach is based on the work on graph transformations by Heckel et al. [11].

WebML follows an MDD approach for mapping modeling elements of WebML to architecture components of MVC2, which can be transformed into components for different platforms [5]. OO-H [10] supports a transformation-based construction of a presentation model based on modeling elements of the navigation model and code generation based on the conceptual, navigation and presentation models. Hera – an approach centered on the Semantic Web–RDF technology – applies MDD to the creation of a model for data integration [32].

WebSA is an approach that focus on architectural models and is partially integrated in the UWE process. [21].

6. CONCLUSIONS AND FUTURE WORK

We presented the development process of the UML-based Web Engineering (UWE) approach defined as a model-driven development approach. We outlined all the models and model transformations that comprise the MDD process, focusing on the classification of the model transformations in terms of: MDA transformation type, level of abstraction of the rule definition, number of source models, involvement of marking models, transformation techniques and execution type.

We used different specification techniques for the transformations like ATL, QVT, graph transformations, and hard-coded in Java. The need to redefine some of them, e.g. those that are hard coded in a design tool.

By the time this paper is written, the main problem still is the tool support for model transformations. A detailed analysis of the requirements of such tools is beyond the scope of this paper. Others like QVT are very new approaches, which still provide insufficient tool support. We plan to concentrate on the most promising and adequate approach and those that provide a user-friendly tool environment. For example, we plan to use the AGG [30] and the apache struts technology (`www.apache.org`) to produce results that can then be integrated with the tool environment HUGO/RT [15] for model checking purposes. We will also use our research results in this area for the automatic generation of test cases.

ACKNOWLEDGMENTS

We would like to thank the Web Engineering team of the LMU, which contributed to the improvement of UWE, in particular to Alexander Knapp, Gefei Zhang, Andreas Kraus, Rolf Hennicker and Hubert Baumeister. We also thank María José Escalona of the University of Seville, and Santiago Meliá and Cristina Cachero of the University of Alicante for their collaborations in other works related to UWE. We grateful thank Alexander Knapp and Gefei Zhang for their fruitful comments on a draft version of this paper.

REFERENCES

[1] Amber, S. Agile Model-Driven Development with UML 2.0. Cambridge University Press, 2004.

[2] Baresi, L. and Mainetti, L. Beyond Modeling Notations: Consistency and Adaptability of W2000 Models. In Proc. of SAC'05, ACM Symposium on Applied Computing, Santa Fe, USA, 2005

[3] Baumeister, H., Knapp, A., Koch, N. and Zhang, G. Modelling Adaptivity with Aspects. In 5th International Conference on Web Engineering (ICWE 2005), LNCS 3579, ©Springer Verlag, July 2005.

[4] Bézivin, J.. In Search of a Basic Principle for Model Driven Engineering. UPGRADE Vol V. No. 2, Novótica, April 2004.

[5] Ceri, S., Fraternali, P. and Matera, M. Conceptual Modeling of Data-Intensive Web Applications, IEEE Internet Computing 6(4), July/August, 2002.

[6] Escalona, M. J..and Koch, N. Metamodeling Requirements of Web Systems. In Proc. 2nd International Conference on Web Information System and Technologies, Setubal, Portugal. 2006.

[7] Escalona, M. J., Mejías, M. and Torres, J.. Developing System with NDT & NDT-Tool. Proceeding of the 13th International Conference on Information System Development (ISD'2004).pp. 149-159, Lithuania. 2004.

[8] Engels, G., Hausmann, J-H, Heckel, R. and Sauer, S. Dynamic Meta Modeling: A Graphical Approach to Operational Semantics of Behavioral Diagrams in UML. In Proc. 3th Int. Conference on the Unified Modeling Language (UML 2000), LNCS 1939, Springer Verlag, October 2000.

[9] Frasincar F., Houben, G.J. and Vdovjak R. An RMM-Based Methodology for Hypermedia Presentation Design. In Proc. of Advances in Databases and Information Systems (ADBIS 2001) Vilnius, Lithuania, Springer, LNCS 2151, 2001.

[10] Gomez, J., Cachero, C. and Pastor, O. Extending a Conceptual Modelling Approach to Web Application Design. In Proc. 2nd CaiSE´00, LNCS 1789, Springer Verlag, 2000.

[11] Heckel R. and Lohmann M. Model-based development of Web applications using graphical reaction rules. In Proc. Fundamental Approaches to Software Engineering , Warsaw, Poland, Springer Verlag.

[12] Hennicker, R. and Koch, N. A UML-based Methodology for Hypermedia Design. In A. Evans, S. Stuart, and B. Selic, editors, UML'2000 - The Unified Modeling Language - Advancing the Standard, LNCS 1939, York, England, ©Springer Verlag,Oct. 2000.

[13] Jouault, F., and Kurtev, I.. Transforming Models with ATL. In: Proceedings of the Model Transformations in Practice Workshop at MoDELS 2005, , Jamaica.

[14] Knapp, A., Koch, N., Zhang, G. and, Hassler, H.-M. Modeling Business Processes in Web Applications with ArgoUWE. 7th International Conference on the Unified Modeling Language (UML 2004). LNCS 3273. pp. 69-83. Lisboa, 2004.

[15] Knapp, A., Merz, S. and Rauh, C. Model Checking Timed UML State Machines and Collaborations. In Proc. 7th Int. Symp. Formal Techniques in Real-Time and Fault Tolerant Systems LNCS 2469, Springer, Berlin, 2002.

[16] Knapp, A. and Zhang, G. Model Transformations for Integrating and Validating Web Application Models. In Proc. of Modellierung 2006, Innsbruck, 2006, to appear.

[17] Koch, N. Software Engineering for Adaptive Hypermedia Systems: Reference Model, Modeling Techniques and Development Process. Uni-Druck Verlag, 355 pages, 2001. PhD. Thesis, LMU München.

[18] Koch, N. and Kraus, A.. The expressive Power of UML-based Web Engineering. Second International Workshop on Web-oriented Software Technology (IWWOST02). Spain. June, 2002.

[19] Koch, N., Zhang, G. and Escalona, M. J. Model Transformations from Requirements to Web System Design, 2006, 6th International Conference on Web Engineering, Palo Alto, USA, accepted for publication.

[20] Manifesto for Agile Software Development http://agilemanifesto.org/, last retrieved March 22nd, 2006.

[21] Melía, S., Kraus, A. and Koch, N. MDA Transformations Applied to Web Application Development. In Proc. 5th Int. Conference on Web Engineering (ICWE 2005), Sydney, Australia, LNCS 3579, Springer Verlag, July 2005.

[22] Mellor, S., Scott, K., Uhl, A. and Weise, D.. MDA Distelled; Principles of Model-Driven Architecture, Addison Wesley, 2004.

[23] Object Management Group (OMG). MDA Guide Version 1.0.1. omg/2003-06-01, http://www.omg.org/docs/omg/03-06-01.pdf. June 2003.

[24] Object Management Group (OMG). Meta Object Facility (MOF) Core Specification, v2.0, 2006-01-01, http://www.omg.org/cgi-bin/doc?formal/2006-01-01

[25] Object Management Group (OMG). UML 2 Object Constraint Language (OCL), http://www.omg.org/cgi-bin/doc?ptc/2005-06-06

[26] Object Management Group (OMG). Query QVT-Merge Group, Revised submission for MOF 2.0 Query/Views/ Trans-formations RFP. http://www.omg.org/cgi-bin/apps/doc?ad/04-04-01.pdf, April 2004.

[27] Object Management Group (OMG). Unified Modeling Language (UML): Superstructure, version 2.0. Specification, http://www.omg.org/cgi-bin/doc?formal/05-07-04, July 2005.

[28] Object Management Group (OMG). XML Metadata Interchange (XMI), v2.1, 2005-09-01, http:// www.omg.org/technology/documents/formal/xmi.htm

[29] Schwabe D., Rossi G. Developing Hypermedia Applications using OOHDM. Workshop on Hypermedia Development Process, Methods and Models, Hypertext´98, Pittsburg, USA, 1998.

[30] Taenzer, G. AGG: A Graph Transformation Environment for System Modeling and Validation. In T. Margaria (ed.), Proc. Tool Exihibition at "Formal Methods 2003", Pisa, Italy, September 2003.

[31] Varró, D. and Pataricza A. Generic and Meta-transformations for Model Transformation Engineering. In Proc. 7th Int. Conference on the Unified Modeling Language (UML 2004), LNCS 3273, Springer Verlag, 2004.

[32] Vdovjak, R. and Houben G.J.A Model-Driven Approach for Designing Distributed Web Information Systems. In Proc. of Web Engineering, Fifth International Conference, ICWE 2005, Springer LNCS 3579, 2005.

[33] Zhang, G., Baumeister, H., Koch, N. and Knapp, A.. Aspect-Oriented Modeling of Access Control in Web Applications. In 6th Int. Wsh. Aspect Oriented Modeling (AOM'05), Chicago, 2005.